

SRS

▼ GİRİŞ

1.1 Amaç

Bu doküman, "Learning LLMs with LLMs" platformunun yazılım gereksinimlerini tanımlar. Platform, yazılım mühendisliği lisans öğrencilerine LLM teorisi ve uygulamalarını, bizzat bir LLM ile diyalog kurarak öğretmeyi amaçlayan AI destekli bir web uygulamasıdır. Doküman, geliştirme ekibi, araştırmacılar (ICERI2026 çalışması için) ve olası değerlendiriciler için ortak bir referans noktası oluşturur.

1.2 Kapsam

Faz 1 (MVP - abstracta kadar olan çalışmanın kapsamı): Tek bir konu modülü (örn. "Transformer Mimarisi") üzerinden üç katmanlı akışın (Teori → Uygulama → Eleştirel Bakış) uçtan uca çalışan bir prototipi, temel learning analytics, ve küçük ölçekli pilot kullanıcı testi (pre/post anket).

Faz 2 (Tam kapsam — gelecek çalışma): Tüm LLM müfredatının çoklu modül olarak kapsanması, gelişmiş kişiselleştirme (takılma noktası tespiti ile otomatik derinleştirme), tam learning roadmap arayüzü, geniş ölçekli kullanıcı çalışması.

1.3 Tanımlar ve Kısaltmalar

Terim	Açıklama
LLM	Large Language Model
SRS	Software Requirements Specification
Mastery Learning	Öğrencinin bir konuyu gerçekten anlamadan bir sonrakine geçemediği, ama zorla da tutulmadığı öğretim yaklaşımı
Dialogic Learning	Bilginin tek yönlü değil, diyalog yoluyla aktif inşa edildiği öğrenme modeli
Learning Analytics	Öğrenci davranış verisinin toplanıp kişiselleştirme/iyileştirme için analiz edilmesi
Active Session Time	Kullanıcının pasif (etkileşimsiz) geçen süresinin hariç tutulduğu, gerçek etkileşim süresi

▼ GENEL AÇIKLAMA

2.1 Ürün Perspektifi

Platform, bağımsız bir web uygulaması olarak geliştirilecektir. Sistem, bir LLM sağlayıcısının API'sine bağlanarak öğrenciyle diyalog kurar; kendi LLM'ini eğitmez, var olan bir modeli "öğretmen/rehber" rolünde kullanır.

2.2 Ürün Fonksiyonları

Faz 1 (MVP):

- Tek konu modülü (Faz 1 için RAG seçilmiştir) üzerinden üç katmanlı akışın (Teori → Uygulama → Eleştirel Bakış) uçtan uca çalışması
- LLM ile serbest metin diyalogu (Öğrenci kendi sorusunu kendi kelimeleriyle yazıyor, önceden tanımlanmış sorular içerisinden sorması gerekmiyor. LLM bu sorulara açık uçlu, doğal dilde cevap veriyor. Diyalog akışı önceden belirlenmemiş, öğrenci nereye götürürse oraya gidiyor. → Dialogic Learning)
- Katman sonu quiz + mastery learning akışı (yanlışta farklı anlatımla geri dönüş)
- Temel etkileşim verisi kaydı (mesajlar, deneme sayısı, quiz skoru, aktif süre)
- Pre/post anket uygulaması
- Araştırmacı veri erişimi: Ayrı bir arayüz geliştirilmeyecek; SQLite veritabanı dosyasına doğrudan (VSCode SQLite eklentisi veya DB Browser for SQLite gibi araçlarla) erişilerek sorgulama ve export yapılacaktır. Bu nedenle veri modeli, kod yazmadan okunabilir/sorgulanabilir olacak şekilde tasarlanacaktır.
- Basit kimlik tanımlama (isim/öğrenci no, şifresiz) ile oturumlar arası eşleştirme

Faz 2 (Gelecek Çalışma):

- Tablo 2.2.1'deki diğer 6 modülün eklenmesi
- Otomatik "takılma noktası tespiti" ile kişiselleştirilmiş derinleştirme (PDF'teki learning analytics fikri)

- Tam Learning Roadmap arayüzü (kenar çubuğunda ilerleme görselleştirmesi)
- Çoklu öğrenci eşzamanlı kullanım için ölçeklenebilirlik iyileştirmeleri
- Tam giriş/kayıt sistemi (kullanıcı adı + şifre, hesap yönetimi, oturum güvenliği)

2.2.1 Müfredat Modül Haritası

#	Modül	Faz	Kapsadığı Katmanlar
1	Transformer Mimarisi & Attention Mekanizması	Faz 2	Teori
2	Tokenization & Training Süreci	Faz 2	Teori
3	Prompt Engineering	Faz 2	Uygulama
4	API Kullanımı & Entegrasyon	Faz 2	Uygulama
5	Fine-tuning Kavramları	Faz 2	Uygulama
6	RAG (Retrieval-Augmented Generation)	Faz 1 (MVP)	Teori + Uygulama + Eleştirel (tek modülde üç katman da işlenecek)
7	Hallucination, Bias ve Etik Sınırlar	Faz 2	Eleştirel Bakış

2.3 Kullanıcı Sınıfları ve Özellikleri

Öğrenci (yazılım/bilgisayar mühendisliği lisans öğrencisi, LLM ön bilgisi değişken). MVP'de öğrenci kimliği şifresiz, basit bir alanla (isim/öğrenci no) tanımlanır; bu bir hesap sistemi değildir, sadece veri eşleştirme amaçlıdır. Faz 2'de gerçek bir kimlik doğrulama sistemine geçilecektir.

Araştırmacı, platformun bir kullanıcı sınıfı değildir; toplanan veriye sistem dışından (veritabanı dosyası üzerinden) erişir

2.4 İşletim Ortamı

Faz 1 (MVP):

- İstemci: Güncel masaüstü tarayıcı, responsive ama mobil-optimize değil
- Sunucu: Python 3.11+ / FastAPI
- Veritabanı: SQLite
- LLM API: YAZILACAK

Faz 2:

- Mobil tarayıcı tam optimizasyonu
- Veritabanı: Eşzamanlı kullanıcı sayısı artarsa PostgreSQL'e geçiş

2.5 Tasarım ve Uygulama Kısıtları

- 18 günlük süre nedeniyle MVP tek modülle (RAG) sınırlıdır
- LLM API maliyeti ve rate limit'ler göz önünde bulundurulmalı; pilot test sırasında istek sayısı izlenmeli
- Pilot grup küçük (~10-20 öğrenci) varsayıldığından yüksek eşzamanlı yük optimizasyonu MVP'de yapılmayacak

▼ FUNCTIONAL REQUIREMENTS

3.1 Modül Yapılandırma Çerçevesi

ID	Gereksinim
FR-1.1	Sistem, her modülü (teori metni, uygulama görev tanımı, eleştirel tartışma promptu, quiz soruları) tutarlı bir yapılandırma formatında (örn. JSON/YAML) tanımlamalı; yeni modül eklemek kod değişikliği değil, yeni bir yapılandırma dosyası eklemek anlamına gelmelidir.
FR-1.2	MVP kapsamında sisteme tek bir modül (RAG) tanımlanacak; çerçeve, Faz 2'de Bölüm 2.2.1'deki diğer modüllerin eklenmesine doğrudan izin verecek şekilde tasarlanmalıdır.
FR-1.3	Tutor Agent, görev anında aktif modülün yapılandırma dosyasından sistem promptunu dinamik olarak oluşturmalı; konuşma, ilgili modülün kapsamı dışına taşmamalıdır.

3.2 Öğrenci Kimliği ve Oturum Eşleştirme

ID	Gereksinim
FR-2.1	Sistem, öğrenci platforma ilk girdiğinde basit bir kimlik bilgisi (isim ve/veya öğrenci numarası) almalı; şifre/hesap oluşturma gerektirmemelidir (MVP kapsamı).
FR-2.2	Bu kimlik bilgisi, pre-anket, öğrenme oturumu verileri ve post-anket arasında eşleştirme anahtarı olarak kullanılmalıdır.
FR-2.3	(Faz 2) Sistem, kullanıcı adı + şifre tabanlı tam bir kimlik doğrulama mekanizmasına geçmeli; mevcut basit kimlik alanları yeni sisteme taşınabilir olmalıdır.

3.3 Diyalog Tabanlı Öğrenme Motoru

ID	Gereksinim
FR-3.1	Sistem, aktif modülün Teori katmanı için modül yapılandırma dosyasında önceden tanımlanmış, sabit bir giriş anlatımı göstermeli; bu anlatım Tutor Agent tarafından dinamik üretilmemelidir. Giriş gösterildikten sonra sistem, serbest metin diyaloga açılmalıdır.
FR-3.2	Öğrenci kendi sorusunu kendi cümleleriyle yazabilmeli; LLM, modülün içeriğine bağlı kalarak açık uçlu, bağlama uygun cevap üretmelidir.
FR-3.3	Sistem, her diyalog mesajını (öğrenci sorusu + LLM cevabı + modül ID'si), zaman damgasıyla birlikte kaydetmelidir.
FR-3.4	Teori katmanı tamamlandığında sistem, aynı modülün Uygulama katmanına otomatik geçiş sunmalıdır.
FR-3.5	Tutor Agent, modülün kapsamı dışında bir soru/konuyla karşılaşır, öğrenciyi nazikçe aktif modülün konusuna yönlendirmeli, alakasız konularda derinlemesine yanıt üretmemelidir.

3.4 Uygulama Katmanı

ID	Gereksinim
FR-4.1	Sistem, aktif modüle özgü, önceden tanımlanmış bir uygulama görevini LLM rehberliğinde, adım adım kurdurmalıdır.
FR-4.2	Öğrenci, görev sırasında LLM'e soru sorabilmeli, takıldığı noktada yardım isteyebilmelidir.
FR-4.3	Sistem, görevin tamamlanıp tamamlanmadığını takip etmelidir.

3.5 Eleştirel Bakış Katmanı

ID	Gereksinim
FR-5.1	Sistem, aktif modülün sınırlarını ve potansiyel risklerini (örn. RAG için halüsinasyon, retrieval kalitesine bağımlılık) tartışan bir diyalog başlatmalıdır.
FR-5.2	Öğrenci, LLM ile bu sınırlar üzerine serbestçe tartışabilmelidir.

3.6 Quiz ve Mastery Learning Mekanizması

ID	Gereksinim
FR-6.1	Her katman sonunda sistem bir quiz sunmalı: çoğunlukla çoktan seçmeli sorular + bir adet açık uçlu soru.
FR-6.2	Çoktan seçmeli sorular otomatik (sabit doğru cevaba göre) puanlanmalıdır.
FR-6.3	Açık uçlu soru, LLM tarafından değerlendirilip bir başarı/başarısızlık ya da kısmi puan üretmelidir.
FR-6.4	Öğrenci quiz'i geçemezse, sistem "geri dönmek ister misin?" seçeneği sunmalıdır.
FR-6.5	Geri dönmeyi seçerse, sistem aynı konuyu farklı bir anlatımla (daha basit, gerçek dünya örnekli) sunmalı, öğrenciye soru sorma imkânı tanımalı, "şimdi anladın mı?" kontrolü yapmalı ve benzer bir soru ile yeniden ölçmelidir.
FR-6.6	Geri dönmek istemezse, sistem doğrudan bir sonraki katmana geçmelidir.
FR-6.7	Sistem, öğrencinin kaç kez geri döndüğünü ve kaç deneme yaptığını kaydetmelidir.

3.7 Learning Analytics

ID	Gereksinim
FR-7.1	Sistem, öğrencinin hangi konularda LLM'e daha çok soru sorduğunu (mesaj sayısı bazında) kaydetmelidir.
FR-7.2	Sistem, her katmandaki ilerleme durumunu (tamamlandı/devam ediyor/bırakıldı) kaydetmelidir.
FR-7.3	Sistem, aktif oturum süresini Bölüm 1.3'te tanımlanan kurala göre (iki etkileşim arası 2 dakikadan kısaysa aktif, uzunsa pasif sayılır)

ID	Gereksinim
	hesaplamalı ve kaydetmelidir.

3.8 Pre/Post Anket Modülü

ID	Gereksinim
FR-8.1	Sistem, öğrenci platforma ilk girdiğinde bir ön-anket sunmalı ve tamamlanmadan platforma erişime izin vermemelidir.
FR-8.2	Sistem, öğrenci tüm katmanları tamamladığında bir son-anket sunmalıdır.
FR-8.3	Anket cevapları, FR-2.1'deki kimlik bilgisiyle ilişkilendirilerek kaydedilmelidir.

▼ NON-FUNCTIONAL REQUIREMENTS

4.1 Performans

ID	Gereksinim
NFR-1.1	LLM'den dönen cevabın ilk kısmının (streaming varsa) öğrenciye 3 saniye içinde gösterilmeye başlanması hedeflenir.
NFR-1.2	Sistem, MVP kapsamında en az 20 eşzamanlı öğrenciyi performans kaybı olmadan desteklemelidir (pilot grup büyüklüğüne göre).

4.2 Güvenlik ve Gizlilik

ID	Gereksinim
NFR-2.1	Toplanan öğrenci verisi (diyalog kayıtları, anket cevapları) yalnızca araştırma amacıyla kullanılmalı, üçüncü taraflarla paylaşılmamalıdır.
NFR-2.2	Pilot katılımcılardan informed consent alınmalı, bu süreç platforma girişten önce tamamlanmalıdır.
NFR-2.3	LLM API anahtarı gibi hassas bilgiler ortam değişkenlerinde (environment variables) tutulmalı, kod içine gömülmemelidir.

4.3 Kullanılabilirlik (Usability)

ID	Gereksinim
NFR-3.1	Arayüz, teknik olmayan bir öğrencinin de (LLM ön bilgisi az olsa bile) ek eğitim almadan kullanabileceği kadar basit olmalıdır.

ID	Gereksinim
NFR-3.2	Diyalog arayüzü, öğrencinin yazdığı mesajı ve LLM cevabını net biçimde ayırt edilebilir (örn. farklı renk/hizalama) göstermelidir.
NFR-3.3	Sistem, hata durumlarında (örn. LLM API yanıt vermezse) öğrenciye anlaşılır bir hata mesajı göstermeli, teknik detay (stack trace vb.) sızdırmamalıdır.

4.4 Güvenilirlik (Reliability)

ID	Gereksinim
NFR-4.1	Öğrencinin ilerlemesi (hangi katmanda, hangi modülde olduğu) her adımda kaydedilmeli; tarayıcı kapanıp yeniden açılrsa bile kaldığı yerden devam edebilmelidir.
NFR-4.2	LLM API'den hata dönerse sistem en az 1 kez otomatik yeniden deneme yapmalıdır.

4.5 Taşınabilirlik (Portability)

ID	Gereksinim
NFR-5.1	Sistem güncel masaüstü tarayıcılarda sorunsuz çalışmalıdır.
NFR-5.2	Faz 2'de farklı bir LLM sağlayıcısına geçiş gerekirse, API entegrasyon katmanı bu değişikliği minimum kod değişikliğiyle desteklemelidir.

▼ SİSTEM MİMARİSİ

5.1 Genel Mimari Yaklaşım

Platform, üç katmanlı (3-tier) basit bir mimariyle tasarlanacaktır: İstemci (Frontend) → Uygulama Sunucusu (Backend) → Veri/LLM Katmanı. MVP kapsamında mikroservis mimarisi gibi karmaşık yapılar tercih edilmemiştir; tek bir backend süreci yeterlidir.

5.2 Bileşenler

Bileşen	Teknoloji	Sorumluluk
Tutor Agent	LLM API, özel sistem promptu	Teori/Uygulama/Eleştirel katmanlarında öğrenciyle diyalog kurar; aktif modülün içeriğine sadık kalır, konuşma geçmişini bağlam olarak kullanır

Bileşen	Teknoloji	Sorumluluk
Evaluator Agent	LLM API, ayrı sistem promptu, düşük temperature	Açık uçlu quiz cevabını değerlendirir (FR-6.3), tutarlı/objektif puanlama hedefler
Frontend	React	Diyalog arayüzü, quiz ekranları, anket formları, ilerleme göstergesi
Backend	Python / FastAPI	Akışın kontrolü (mastery learning mantığı FR-6.4–6.6), hangi agent'ın ne zaman çağrılacağına karar verme, veri kaydı
Veritabanı	SQLite	Öğrenci kimliği, diyalog kayıtları, quiz sonuçları, anket cevapları, analytics verisi
Modül Yapılandırma Dosyaları	JSON (dosya sistemi)	Her modülün teori/uygulama/eleştirel/quiz içeriğini tanımlar

5.3 Veri Akışı (Yüksek Seviye)

1. Öğrenci tarayıcıdan platforma girer → basit kimlik bilgisi alınır (FR-2.1) → pre-anket sunulur
2. Backend, aktif modülün yapılandırma dosyasını okur, ilgili Teori içeriğini frontend'e gönderir
3. Öğrenci mesaj yazar → frontend, backend'e istek atar → backend, LLM API'sine bağlamla (modül içeriği + konuşma geçmişi) birlikte istek gönderir → cevap öğrenciye döner, eşzamanlı olarak veritabanına kaydedilir
4. Katman sonunda quiz sunulur → cevaplar backend'de değerlendirilir (çoktan seçmeli: kural bazlı; açık uçlu: LLM bazlı) → sonuç mastery learning akışını (FR-6.4–6.6) tetikler
5. Tüm katmanlar tamamlanınca post-anket sunulur, oturum kapanır

5.4 LLM Entegrasyon Yaklaşımı

Backend, ihtiyaca göre iki ayrı agent'ı çağırır:

- **Tutor Agent çağırısı:** Bağlam = (a) aktif modülün ilgili katman içeriği, (b) oturumdaki önceki mesajlar, (c) öğrencinin son mesajı → çıktı: doğal dilde cevap, öğrenciye gösterilir.

- **Evaluator Agent çağrısı:** Bağlam = (a) açık uçlu sorunun beklenen cevap kriterleri (modül yapılandırmasında tanımlı), (b) öğrencinin verdiği cevap → çıktı: yapılandırılmış sonuç (örn. `{"basarili": true/false, "geri_bildirim": "..."}`), backend bunu mastery learning akışına girdi olarak kullanır.

Açık nokta: Konuşma geçmişinin ne kadarının (kaç mesaj/token) bağlama dahil edileceği netleştirilmeli - pratikte maliyet ve tutarlılık dengesi gözetilmeli.

Uygulama katmanı notu: FR-4.1 kapsamında, Uygulama katmanında çalışırken Tutor Agent'a bir **araç (tool/function calling) erişimi** tanımlanır ("doküman ara" gibi) böylece öğrenci sadece RAG'i konuşarak değil, Tutor Agent aracılığıyla gerçek bir retrieval işlemini tetikleyerek deneyimler.

Faz 2 notu: Mimaride, konuşma loglarını analiz edip kişiselleştirme sinyali üreten bir **Personalization Agent** için yer ayrılmıştır (bkz. Bölüm 2.2, "takılma noktası tespiti"); Faz 1'de geliştirilmeyecektir.

5.5 Modül Bazlı Tutor Agent Örneklemesi (yeni)

Tutor Agent, tek bir kod tabanına sahip genel bir roldür; ancak her çağrıldığında, aktif modülün yapılandırma dosyasından (FR-1.1) dinamik olarak oluşturulan modüle özgü bir sistem promptuyla çalışır. Bu sayede:

- Her modül, kendi "tutor kimliğine" sahip olur (örn. RAG Tutor, yalnızca RAG'in teori/uygulama/eleştirel içeriği ve terminolojisiyle sınırlı konuşur)
- Konuşmanın modül kapsamı dışına kayması (scope drift) azaltılır
- Faz 2'de yeni modül eklendiğinde (Bölüm 2.2.1'deki diğer 6 modül), otomatik olarak o modüle özgü bir tutor davranışı ortaya çıkar, agent kodunda değişiklik gerekmez, sadece yeni modül yapılandırma dosyası eklenir.

▼ VERİ MODELİ

6.1 Varlıklar (Entities) — Genel Bakış

Varlık (Tablo)	Açıklama	İlgili Gereksinim
<code>students</code>	Basit kimlik bilgisi	FR-2.1
<code>sessions</code>	Giriş-çıkış aralığı, aktif süre	FR-7.3
<code>modules</code>	Yapılandırma dosyasından okunan modül tanımı	FR-1.1

Varlık (Tablo)	Açıklama	İlgili Gereksinim
<code>dialogue_messages</code>	Öğrenci sorusu + Tutor Agent cevabı	FR-3.3
<code>application_tasks</code>	Uygulama görevi adım ilerlemesi	FR-4.3
<code>quiz_results</code>	Çoktan seçmeli + açık uçlu sonuçlar, deneme sayısı	FR-6.2, FR-6.3, FR-6.7
<code>revisit_logs</code>	Mastery learning'de "geri dön" seçimleri	FR-6.7
<code>survey_responses</code>	Pre/post anket cevapları	FR-8.3

6.2 Alan Seviyesi Şema

`students`

Alan	Tip	Açıklama
id	INTEGER PK	Otomatik artan kimlik
name	TEXT	Öğrenci adı
student_no	TEXT (nullable)	Öğrenci numarası (isteğe bağlı)
created_at	TIMESTAMP	Kayıt zamanı

`sessions`

Alan	Tip	Açıklama
id	INTEGER PK	
student_id	INTEGER FK → students.id	
started_at	TIMESTAMP	Oturum başlangıcı
last_activity_at	TIMESTAMP	Son etkileşim zamanı (FR-7.3 hesabı için)
active_duration_seconds	INTEGER	Hesaplanan toplam aktif süre
ended_at	TIMESTAMP (nullable)	Oturum bitişi

`modules`

Alan	Tip	Açıklama
id	INTEGER PK	
code	TEXT	Örn. <code>"rag"</code>

Alan	Tip	Açıklama
name	TEXT	Görünen ad, örn. "RAG (Retrieval-Augmented Generation)"
phase	INTEGER	1 veya 2 (bkz. Bölüm 2.2.1)
config_path	TEXT	Yapılandırma dosyasının yolu (FR-1.1)

dialogue_messages

Alan	Tip	Açıklama
id	INTEGER PK	
session_id	INTEGER FK → sessions.id	
module_id	INTEGER FK → modules.id	
layer	TEXT	theory / application / critical
sender	TEXT	student / tutor_agent
content	TEXT	Mesaj içeriği
created_at	TIMESTAMP	

application_tasks

Alan	Tip	Açıklama
id	INTEGER PK	
session_id	INTEGER FK → sessions.id	
module_id	INTEGER FK → modules.id	
step_number	INTEGER	Görev adım sırası
status	TEXT	not_started / in_progress / completed
completed_at	TIMESTAMP (nullable)	

quiz_results

Alan	Tip	Açıklama
id	INTEGER PK	
session_id	INTEGER FK → sessions.id	

Alan	Tip	Açıklama
module_id	INTEGER FK → modules.id	
layer	TEXT	Hangi katman sonu quiz'i
mcq_score	REAL	Çoktan seçmeli puanı (FR-6.2)
open_ended_score	REAL (nullable)	Evaluator Agent puanı (FR-6.3)
open_ended_feedback	TEXT (nullable)	Evaluator Agent geri bildirimi
attempt_number	INTEGER	Kaçıncı deneme
passed	BOOLEAN	Geçti/kaldı (eşik TBD — FR-6.4)
created_at	TIMESTAMP	

revisit_logs

Alan	Tip	Açıklama
id	INTEGER PK	
session_id	INTEGER FK → sessions.id	
module_id	INTEGER FK → modules.id	
layer	TEXT	
revisited	BOOLEAN	Öğrenci "geri dön" mü seçti (FR-6.4–6.5)
created_at	TIMESTAMP	

survey_responses

Alan	Tip	Açıklama
id	INTEGER PK	
student_id	INTEGER FK → students.id	
survey_type	TEXT	pre / post
question_id	TEXT	Anket sorusu kimliği
answer	TEXT	Cevap (serbest metin veya seçenek kodu)
created_at	TIMESTAMP	

6.3 İlişkiler (Entity Relationships)

- `students` 1—N `sessions` (bir öğrencinin birden fazla oturumu olabilir)
- `sessions` 1—N `dialogue_messages` , `application_tasks` , `quiz_results` , `revisit_logs`
- `modules` 1—N `dialogue_messages` , `application_tasks` , `quiz_results` , `revisit_logs` (her kayıt hangi modüle ait olduğunu taşır)
- `students` 1—N `survey_responses`

6.4 Tasarım Notları

- Tüm tablolarda `created_at` alanı bulunur; bu, hem Learning Analytics (FR-7.x) hem de zaman bazlı analiz (örn. "öğrenci hangi saatte daha aktif") için kullanılabilir.
- Şema, Bölüm 2.3'te kararlaştırıldığı gibi **kod yazmadan SQLite araçlarıyla doğrudan okunabilir** olacak şekilde sade tutulmuştur (gereksiz normalizasyon yapılmamıştır).
- Faz 2'de `students` tablosuna `password_hash` gibi alanlar eklenmesi planlanmaktadır (FR-2.3, tam giriş/kayıt sistemi).

▼ USE CASES

7.1 Use Case Listesi

ID	Use Case	İlgili FR'ler
UC-1	Platforma Giriş ve Pre-Anket Tamamlama	FR-2.1, FR-2.2, FR-8.1
UC-2	Teori Katmanında LLM ile Diyalog Kurma	FR-3.1–3.4
UC-3	Uygulama Görevini Tamamlama (RAG Kurulumu)	FR-4.1–4.3
UC-4	Eleştirel Bakış Tartışması	FR-5.1, FR-5.2
UC-5	Katman Sonu Quiz'i Çözme	FR-6.1–6.3
UC-6	Mastery Learning — Konuya Geri Dönme	FR-6.4, FR-6.5, FR-6.7
UC-7	Mastery Learning — Sonraki Katmana Geçme	FR-6.4, FR-6.6
UC-8	Platformu Tamamlama ve Post-Anket	FR-8.2, FR-8.3

UC-1: Platforma Giriş ve Pre-Anket Tamamlama

- **Aktör:** Öğrenci
- **Ön Koşul:** Yok (öğrencinin platforma ilk girişi)
- **Akış:**
 1. Öğrenci platforma erişir, basit kimlik bilgisini girer (isim ve/veya öğrenci no — FR-2.1)
 2. Sistem, bu bilgiyle bir `students` kaydı oluşturur (yoksa) ve yeni bir `session` başlatır
 3. Sistem, pre-anket sorularını sırayla gösterir
 4. Öğrenci tüm soruları cevaplar; her cevap `survey_responses` tablosuna `survey_type = "pre"` olarak kaydedilir
 5. Anket tamamlanmadan sistem öğrenme içeriğine erişime izin vermez (FR-8.1)
 6. Anket bitince sistem, aktif modülün (RAG) Teori katmanına yönlendirir (UC-2)
- **Alternatif Akış:** Öğrenci daha önce kayıtlıysa (aynı isim/öğrenci no ile tekrar giriyorsa), sistem mevcut `student_id` 'yi eşleştirir ve kaldığı yerden devam ettirir (NFR-4.1 ile bağlantılı) — pre-anket tekrar sorulmaz.

UC-2: Teori Katmanında LLM ile Diyalog Kurma

- **Aktör:** Öğrenci
- **Ön Koşul:** Pre-anket tamamlanmış, RAG modülü Teori katmanı aktif
- **Akış:**
 1. Sistem, RAG'in teorik giriş anlatımını gösterir (önceden hazırlanmış sabit metin)
 2. Öğrenci, diyalog kutusuna serbest metin bir soru yazar
 3. Sistem, Tutor Agent'ı modül bağlamı + konuşma geçmişiyle çağırır
 4. Tutor Agent cevabı öğrenciye gösterilir, mesaj çifti veritabanına kaydedilir
 5. Öğrenci istediği kadar soru sormaya devam eder

6. Öğrenci "devam et" seçeneğiyle Teori katmanını bitirip quiz'e (UC-5) geçer

- **Alternatif Akış:** Tutor Agent, modül kapsamı dışı bir soruya (örn. alakasız bir konu) gelirse, kibarca konuya yönlendirici bir cevap verir

UC-3: Uygulama Görevini Tamamlama (RAG Kurulumu)

- **Aktör:** Öğrenci
- **Ön Koşul:** Teori katmanı ve quiz'i geçilmiş (UC-2, UC-5 tamamlanmış)
- **Akış:**
 1. Sistem, RAG modülünün uygulama görev tanımını (yapılandırma dosyasından) ve ilk adımı gösterir
 2. Tutor Agent, bu katmanda **doküman arama aracına (tool access)** sahiptir (bkz. 5.4)
 3. Öğrenci, Tutor Agent'a görevle ilgili komut/soru yazar (örn. "bu dokümanlarda X ile ilgili ne var?")
 4. Tutor Agent, gerektiğinde aracı çağırarak (retrieval işlemi fiilen tetikleyerek) sonucu öğrenciye gösterir, aynı zamanda süreci açıklar
 5. Öğrenci, görevin ilgili adımını tamamladığını işaretler; sistem `application_tasks.status` alanını günceller
 6. Tüm adımlar tamamlanınca sistem, Eleştirel Bakış katmanına (UC-4) geçişi sunar
- **Alternatif Akış:** Öğrenci bir adımda takılırsa, Tutor Agent'tan yardım isteyebilir (FR-4.2); bu, normal diyalog mekanizmasıyla aynı şekilde işler, görev durumu değişmez.

UC-5: Katman Sonu Quiz'i Çözme

- **Aktör:** Öğrenci
- **Ön Koşul:** İlgili katman (Teori/Uygulama/Eleştirel) tamamlanmış
- **Akış:**
 1. Sistem, ilgili katmanın quiz sorularını gösterir: çoğunlukla çoktan seçmeli + 1 açık uçlu

2. Öğrenci tüm soruları cevaplar, gönderir
3. Sistem, çoktan seçmeli soruları kural bazlı puanlar (FR-6.2)
4. Sistem, açık uçlu cevabı Evaluator Agent'a gönderir; Evaluator Agent yapılandırılmış bir sonuç (başarı durumu + geri bildirim) döner (FR-6.3)
5. Sistem, toplam sonucu `quiz_results` tablosuna kaydeder, `passed` alanını eşik değerine göre belirler (eşik — TBD, FR-6.4)
6. Sonuç "geçti" ise → UC-7 (sonraki katmana geçiş); "kaldı" ise → "geri dönmek ister misin?" sorusu sorulur → UC-6 veya UC-7

UC-6: Mastery Learning — Konuya Geri Dönme

- **Aktör:** Öğrenci
- **Ön Koşul:** Quiz'de eşik altında kalınmış (FR-6.4 tetiklenmiş)
- **Akış:**
 1. Sistem "geri dönmek ister misin?" sorusunu sunar
 2. Öğrenci "evet" seçer
 3. Sistem, Tutor Agent'tan aynı konunun daha basit, gerçek dünya örnekli bir anlatımını ister
 4. Öğrenci yeni anlatımı okur, isterse soru sorar (UC-2 ile aynı diyalog mekanizması)
 5. Sistem "şimdi anladın mı?" kontrolü sorar
 6. Sistem, benzer ama farklı bir soru ile yeniden ölçer (yeni bir `quiz_results` kaydı, `attempt_number` +1)
 7. `revisit_logs` tablosuna bu geri dönüş kaydedilir

▼ DEĞERLENDİRME PLANI

8.1 Değerlendirme Amacı

Bu değerlendirme, platformun öğrencilerin LLM teorisi ve uygulamaları konusundaki bilgi/beceri düzeyinde anlamlı bir değişim yaratıp yaratmadığını ölçmeyi amaçlar

8.2 Pilot Grup

- **Katılımcı profili:** Yazılım/bilgisayar mühendisliği lisans öğrencileri, LLM ön bilgisi değişken
- **Grup büyüklüğü:** 15-20 öğrenci
- **Seçim yöntemi:** [Gönüllü katılım mı, belirli bir ders kapsamında mı? — netleştirilmeli]

8.3 Veri Toplama Araçları

Araç	Ne Zaman	Ölçtüğü
Pre-anket	Platforma ilk girişte	Başlangıç LLM bilgi/farkındalık düzeyi
Post-anket	Tüm katmanlar tamamlandığında	Bitiş LLM bilgi/farkındalık düzeyi
Quiz sonuçları (<code>quiz_results</code>)	Her katman sonu	Anlık öğrenme kazanımı (katman bazında)
Learning analytics (<code>dialogue_messages</code> , <code>revisit_logs</code> , oturum süresi)	Sürekli	Etkileşim deseni, takılma noktaları

8.4 Analiz Yaklaşımı

- **Pre/Post karşılaştırma:** Her öğrencinin pre- ve post-anket puanları `student_id` üzerinden eşleştirilir.
- **Normallik kontrolü:** Fark puanlarının dağılımı Shapiro-Wilk testi ile kontrol edilir.
- **İstatistiksel test:** Veri normal dağılıyorsa eşleştirilmiş t-testi (paired t-test); değilse Wilcoxon signed-rank testi uygulanır.
- **Etki büyüklüğü:** Cohen's d hesaplanarak, anlamlılığın yanı sıra etkinin pratik büyüklüğü de raporlanır.
- **Nitel veri:** Diyalog mesajları, soru sıklığı ve geri dönüş sayıları betimsel istatistiklerle (ortalama, medyan) özetlenir.

8.5 Başarı Kriterleri

"Post-anket puanlarının pre-anket puanlarına göre istatistiksel olarak anlamlı ($p < 0.05$) bir artış göstermesi ve orta/büyük düzeyde bir etki büyüklüğü

(Cohen's $d \geq 0.5$) gözlemlenmesi hedeflenir."

▼ ZAMAN ÇİZELGESİ

Tarih Aralığı	Gün Sayısı	Aşama	Çıktı
21-22 Haz	2 gün	SRS finalizasyonu, RAG modül içeriğinin (teori metni, uygulama görev adımları, quiz soruları) hazırlanması, anket sorularının yazılması	SRS dokümanı, modül yapılandırma dosyası (JSON/YAML), anket soru seti
23-29 Haz	7 gün	Geliştirme: Backend (FastAPI), veritabanı şeması, Tutor/Evaluator Agent entegrasyonu, frontend (React), mastery learning akışı	Çalışan MVP (yerel ortamda)
30 Haz – 1 Tem	2 gün	İç test ve hata düzeltme (developer kendi kendine uçtan uca akışı test eder)	Stabil, test edilmiş prototip
2-4 Tem	3 gün	Pilot test: katılımcı toplama + platformun öğrencilerle çalıştırılması (pre-anket → öğrenme → post-anket)	Ham veri (15-20 öğrenci)
5-6 Tem	2 gün	Veri analizi: normallik kontrolü, t-testi/Wilcoxon, Cohen's d, betimsel istatistikler	Analiz sonuçları
7-8 Tem	2 gün	Abstract yazımı, gözden geçirme	250 kelime altı abstract
9 Tem	—	Son teslim	ICERI2026 submission